Adversa

An adversarial-resistant machine learning workspace for security data

Student	Srivatsa Shashikumar (PES1PGX25CS072)
Programme	Executive M.Tech, Cybersecurity Engineering
Course	AI/ML for Cybersecurity
Built with	Google Antigravity (AI IDE), Python, Streamlit, PyTorch, XGBoost, SHAP
Deliverables	Project code, datasets, screenshots, demo video, key prompts, README, this report

1. Aim

The aim of this project was to use an AI-assisted code editor to build a complete, working application that does real analysis on a public cybersecurity dataset, guided mainly by prompts I wrote and then reviewed, corrected and tested myself.

The result is Adversa. It is a general workspace for tabular security data. A user uploads a file, chooses what to predict (or nothing, for unlabelled data), and the app trains several models, attacks the best one with adversarial inputs, explains its decisions, and turns the findings into detection rules mapped to MITRE ATT&CK. Phishing detection is the main worked example, and the same pipeline is also shown on an authentication log.

2. The problem

Security detectors are usually judged on a clean test set. An attacker does not behave like a clean test set. They change their inputs a little at a time until the detector misses them. A model that scores 95% in the lab can fail quickly against that kind of pressure, and a model that is accurate but cannot explain itself is hard for an analyst to trust.

So Adversa asks three questions of any detector it builds:

1. **Which model detects best** on this data, measured by how many attacks it catches and how often it raises a false alarm?
2. **How easily is it evaded?** If the inputs are nudged the way an attacker would nudge them, how much of its detection ability survives?
3. **Why does it flag what it flags,** and can that be turned into a rule a defender can deploy?

3. Datasets

3.1 Main dataset: phishing URLs

File: data/public/phishing_urls_10k.csv. It has 10,000 rows and 18 columns: 5,000 phishing and 5,000 legitimate URLs. Each row has a domain name, 16 binary features (for example whether the URL contains an IP address, an @ symbol, a very long length, a redirect, an iframe, or a young domain) and a Label column (1 = phishing, 0 = legitimate).

Item	Detail
Source	Shreya Gopal, Phishing Website Detection by Machine Learning Techniques, GitHub repository, file DataFiles/5.urldata.csv
Phishing URLs	5,000 random URLs from PhishTank (phishtank.com)
Legitimate URLs	5,000 random URLs from the University of New Brunswick URL 2016 dataset (unb.ca/cic/datasets/url-2016.html)
Feature definitions	Follow the UCI Phishing Websites dataset (archive.ics.uci.edu/ml/datasets/Phishing+Websites)
Known issue	Many repeated rows. The app's cleaner removes them (5,626 duplicates, leaving 4,374 unique records).

3.2 Second dataset: authentication log

To show that the tool is not tied to phishing, I also ran the app's bundled sample SSH authentication log (auth_syslog.log, 250 lines, 28 KB). It is semi-structured text from hosts such as core-router-01, vpn-gateway-01, dmz-firewall-02 and dc-primary.internal, with accepted public-key logins and failed passwords for an invalid user. The app parses the raw log into 11 columns (log index, timestamp, host, process id, raw message, source IP, port, action, status and others). This is a small sample file used to demonstrate a different format and a different kind of task. It is not a real public dataset.

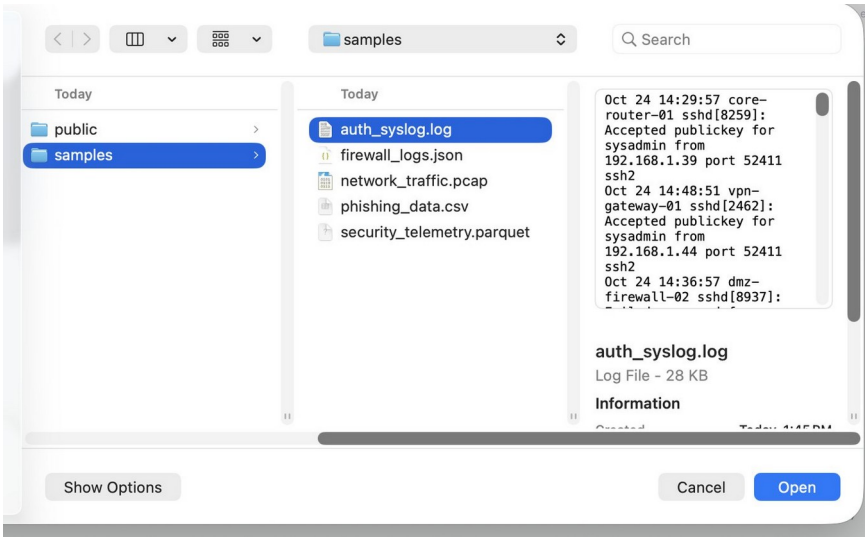


Figure 1. Choosing a log file in the upload dialog. The app also accepts CSV, JSON, PCAP and Parquet.

4. How it was built

I built the application with Google Antigravity, writing prompts that named the stages and outputs I wanted, then reviewing what came back. The pipeline is split into small modules so each stage can be tested on its own.

Stage	Where	What it does
Ingestion	src/parsers	Reads CSV, JSON, logs, PCAP and Parquet through one factory. A buffer validator rejects unsafe or oversized files before parsing.

Stage	Where	What it does
Privacy	src/security	Masks IP addresses and personal data in memory before any analysis. Nothing is written to disk.
Cleaning	src/cleaners	Removes duplicate rows, drops columns with no variance, fills missing values.
Model arena	src/models	Trains and compares classical models and neural networks. Chooses classification, regression or unsupervised anomaly detection from the target column.
Robustness	src/adversarial	Perturbs the inputs (FGSM-style and Laplacian noise) and measures how much detection performance survives.
Explainability	src/xai	Uses SHAP to rank the features behind each decision and writes a plain-English summary.
Rules	src/mitre	Maps findings to MITRE ATT&CK and writes YARA, Snort and iptables rules. Uses an LLM when an API key is set, and an offline generator when it is not.
Verification	tests	Ten automated checks, one per stage, run from the app and with pytest.

The interface is a Streamlit dashboard with a left settings sidebar, four summary cards across the top, and five tabs that follow the pipeline. It has dark and light themes. The sidebar also holds the API key settings for Gemini, Anthropic, OpenAI and other compatible providers, which are optional.

5. Walkthrough and results

5.1 Ingestion and privacy

The first tab loads a file and shows how many records and features it contains. In the authentication-log run the app loaded 250 records with 11 columns. The privacy switch was on, so IP addresses were replaced by tokens such as INTERNAL_HOST_001 before anything was displayed or analysed. The Ingestion status panel reports 66 masked entities.

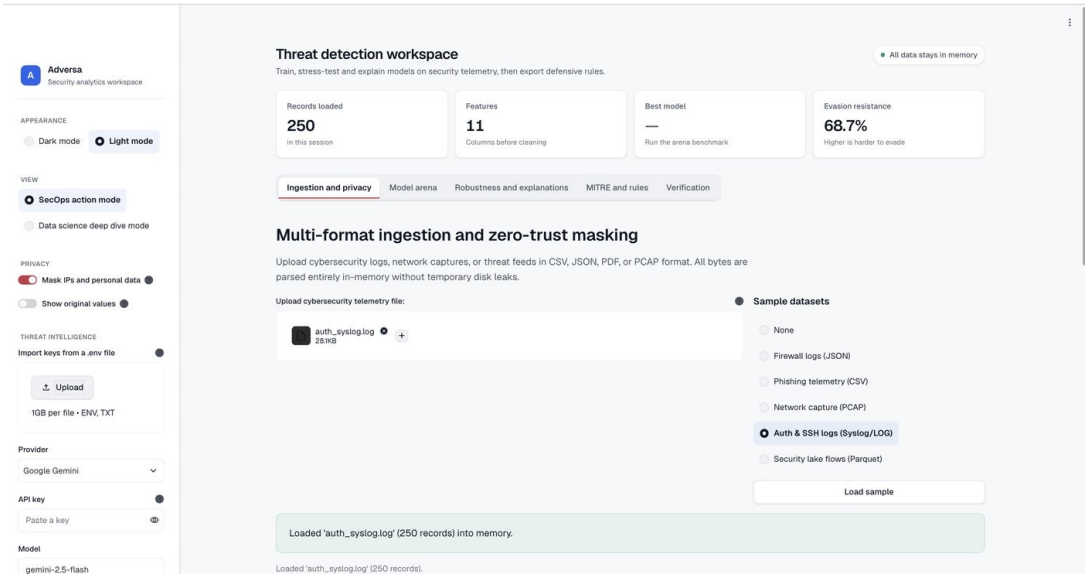
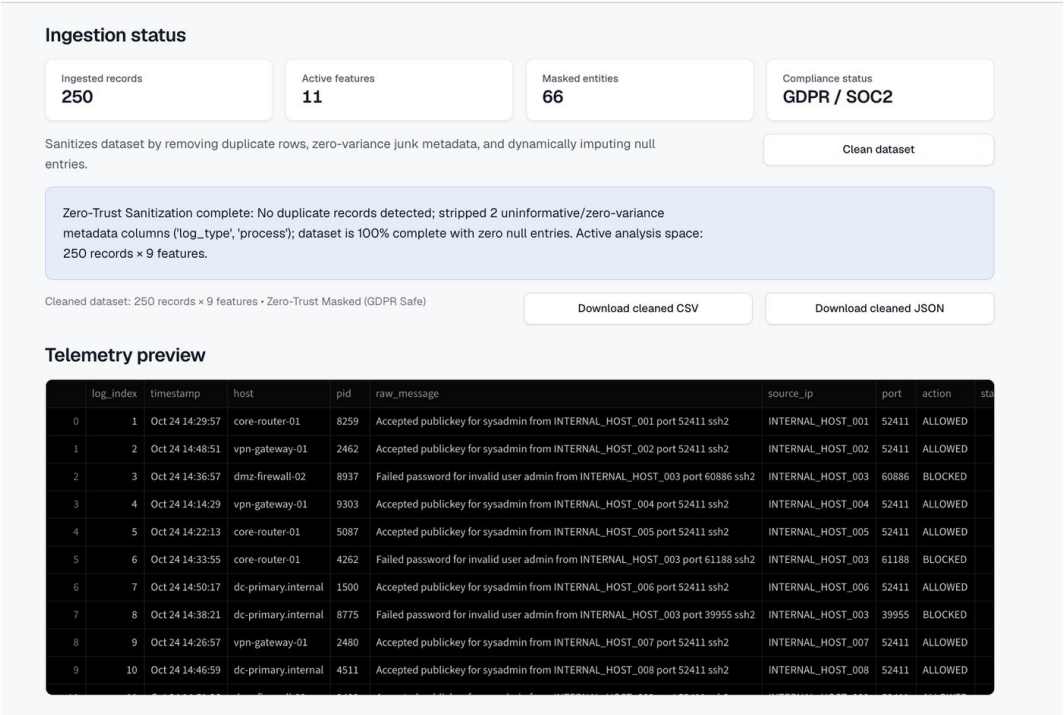


Figure 2. Authentication log loaded: 250 records, 11 features, privacy masking on, light theme.

5.2 Cleaning

Cleaning is a button, so the user sees exactly what was changed. On the authentication log it found no duplicate rows, removed two columns that never change (log_type and process) because they carry no information, and left 250 records by 9 features with no missing values.

On the phishing file the effect is much larger. Of the 10,000 rows, 5,626 were repeats, leaving 4,374 unique records. This matters. If duplicates are left in, the same row can land in both the training and test sets, and the model is partly scored on data it has already seen. That inflates every accuracy figure.



	log_index	timestamp	host	pid	raw_message	source_ip	port	action	sta
0	1	Oct 24 14:29:57	core-router-01	8259	Accepted publickey for sysadmin from INTERNAL_HOST_001 port 52411 ssh2	INTERNAL_HOST_001	52411	ALLOWED	
1	2	Oct 24 14:48:51	vpn-gateway-01	2462	Accepted publickey for sysadmin from INTERNAL_HOST_002 port 52411 ssh2	INTERNAL_HOST_002	52411	ALLOWED	
2	3	Oct 24 14:36:57	dmz-firewall-02	8937	Failed password for invalid user admin from INTERNAL_HOST_003 port 60886 ssh2	INTERNAL_HOST_003	60886	BLOCKED	
3	4	Oct 24 14:14:29	vpn-gateway-01	9303	Accepted publickey for sysadmin from INTERNAL_HOST_004 port 52411 ssh2	INTERNAL_HOST_004	52411	ALLOWED	
4	5	Oct 24 14:22:13	core-router-01	5087	Accepted publickey for sysadmin from INTERNAL_HOST_005 port 52411 ssh2	INTERNAL_HOST_005	52411	ALLOWED	
5	6	Oct 24 14:33:55	core-router-01	4262	Failed password for invalid user admin from INTERNAL_HOST_003 port 61188 ssh2	INTERNAL_HOST_003	61188	BLOCKED	
6	7	Oct 24 14:50:17	dc-primary.internal	1500	Accepted publickey for sysadmin from INTERNAL_HOST_006 port 52411 ssh2	INTERNAL_HOST_006	52411	ALLOWED	
7	8	Oct 24 14:38:21	dc-primary.internal	8775	Failed password for invalid user admin from INTERNAL_HOST_003 port 39955 ssh2	INTERNAL_HOST_003	39955	BLOCKED	
8	9	Oct 24 14:26:57	vpn-gateway-01	2480	Accepted publickey for sysadmin from INTERNAL_HOST_007 port 52411 ssh2	INTERNAL_HOST_007	52411	ALLOWED	
9	10	Oct 24 14:46:59	dc-primary.internal	4511	Accepted publickey for sysadmin from INTERNAL_HOST_008 port 52411 ssh2	INTERNAL_HOST_008	52411	ALLOWED	

Figure 3. Authentication log after cleaning: duplicate check, two zero-variance columns removed, masked preview.

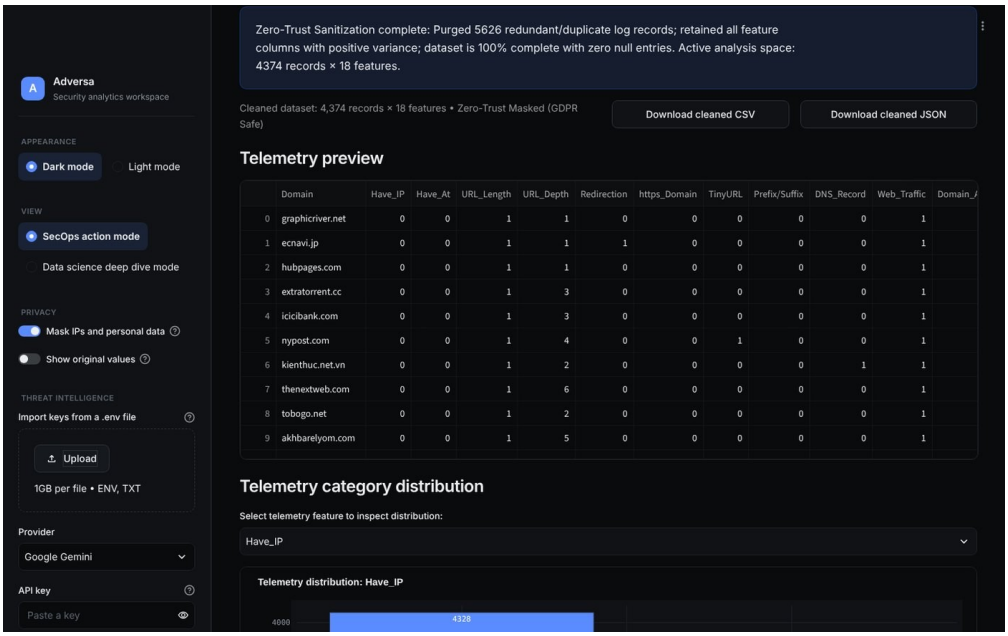


Figure 4. Phishing data after cleaning: 5,626 duplicate rows purged, 4,374 records by 18 features remain.

Below the preview, a category chart shows how values are distributed in any chosen column. For the log it shows the four hosts are roughly balanced (68, 64, 61 and 57 records), which means the data was not dominated by one machine.

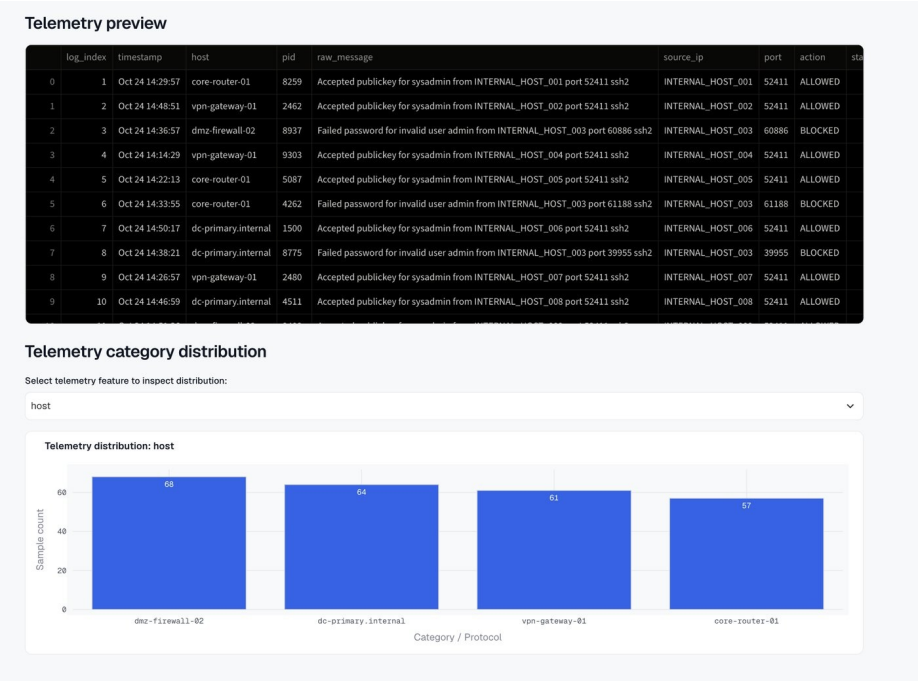


Figure 5. Distribution of the host column in the authentication log.

5.3 Model arena

The arena decides what kind of problem it is from the target column that the user selects. A category target (such as Label) gives classification. A numeric target gives regression. Choosing no target gives unsupervised anomaly detection with a deep autoencoder. In SecOps view the result is shown as a verdict. In data-science view it is shown as a leaderboard, raw metrics and a training-loss curve.

Supervised classification on the phishing data. With Label as the target, the models were trained on the same split and ranked. In the recorded run XGBoost won. It caught 93.5% of phishing sites (this is recall) while wrongly flagging 6.6% of legitimate sites (the false positive rate). Random forest and logistic regression were also trained, and a neural network was part of the comparison. The winner can change slightly from run to run because training involves randomness. An earlier run on the same data was won by a different model.

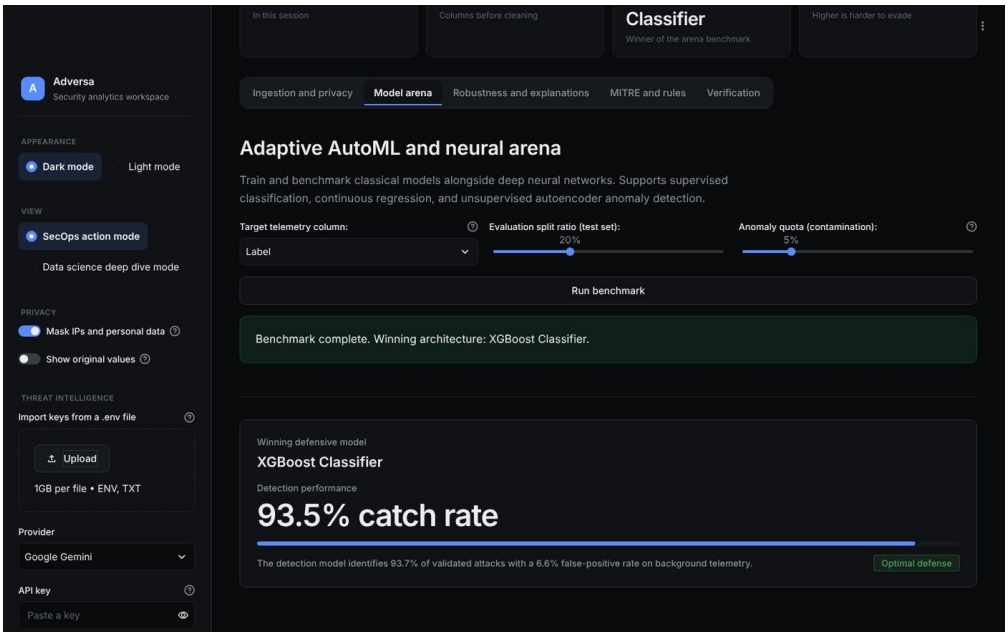


Figure 6. Phishing benchmark in SecOps view: XGBoost wins with a 93.5% catch rate.

Unsupervised anomaly detection on the log. With no target, the app trains an autoencoder, a neural network that learns to rebuild normal log entries. Entries it rebuilds badly are unusual. The user sets an anomaly quota (contamination), here 5%. The autoencoder flagged 13 of 250 entries, with a reconstruction-error threshold of 0.1508. The flagged entries were repeated failed-password attempts for an invalid user from the same masked host (INTERNAL_HOST_003), which is the pattern a brute-force attack leaves. The summary reports the flagged entries deviate by +692.7% from the baseline.

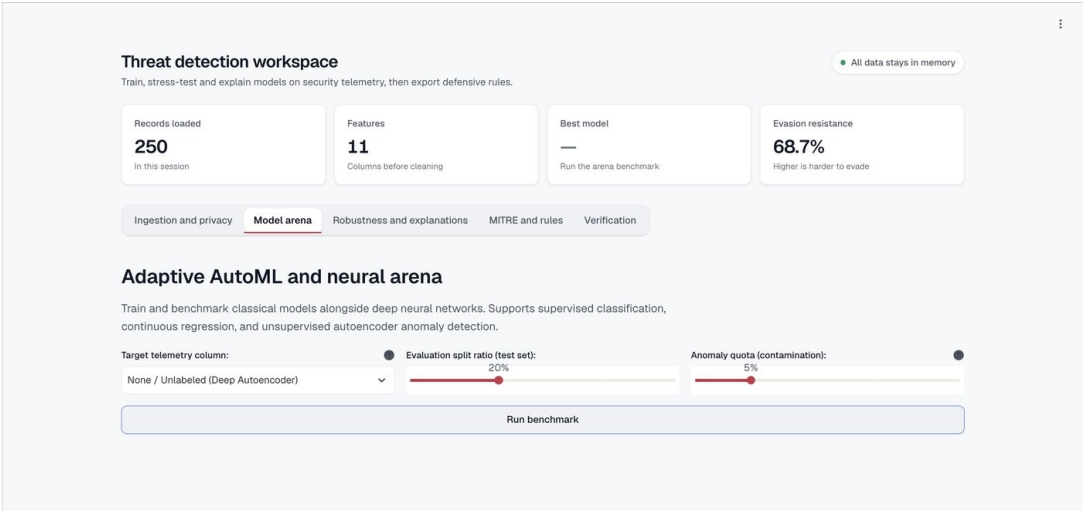


Figure 7. Arena set-up: no target column selected, so the autoencoder is used. Sliders set the test split and anomaly quota.

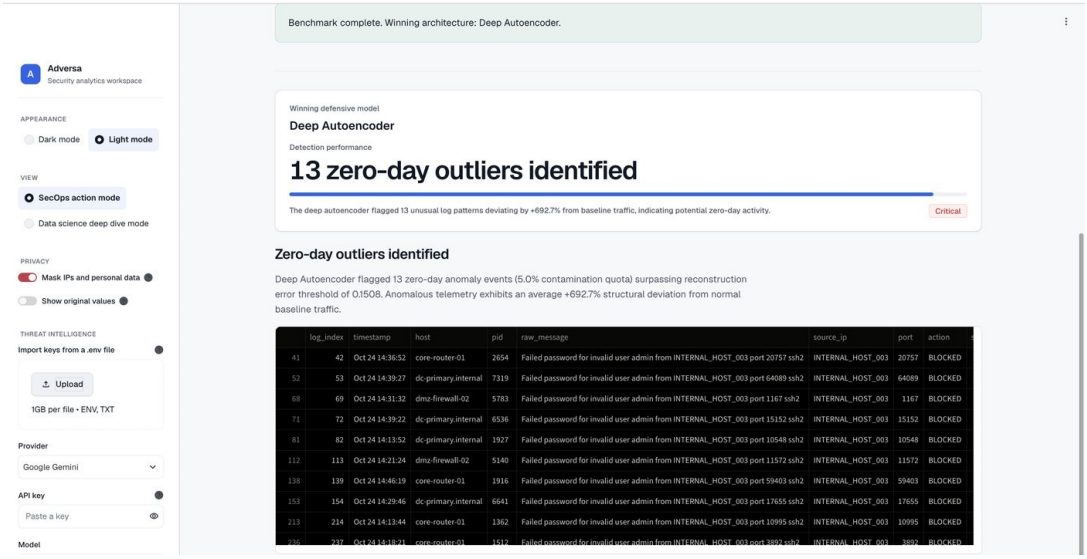


Figure 8. SecOps view: 13 zero-day outliers identified, shown with the log lines that triggered them.

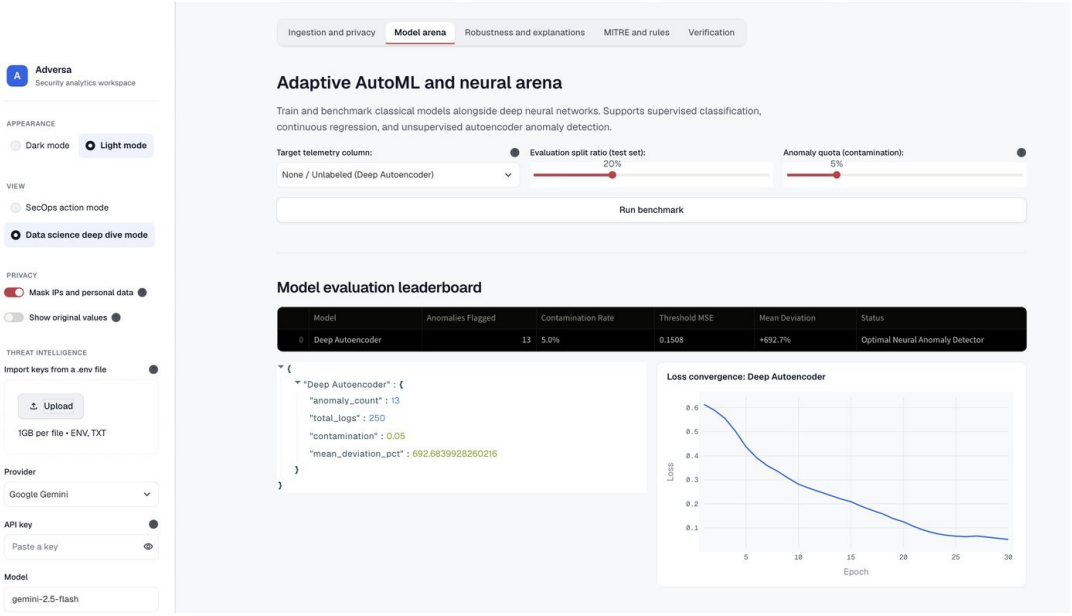


Figure 9. Data-science view: leaderboard, raw metrics and the autoencoder loss curve falling from about 0.6 to 0.05 over 30 epochs.

Regression. To check the numeric mode I selected log_index as the target. Ridge regression ranked first, but with an R-squared of -0.03 (random forest -0.11, neural network -2.22). A negative R-squared means the model is no better than guessing the average. That is the correct outcome, because a row number cannot be predicted from the other columns. I include it only to show that the regression mode runs and that the metrics behave sensibly. It is not a finding about the log.

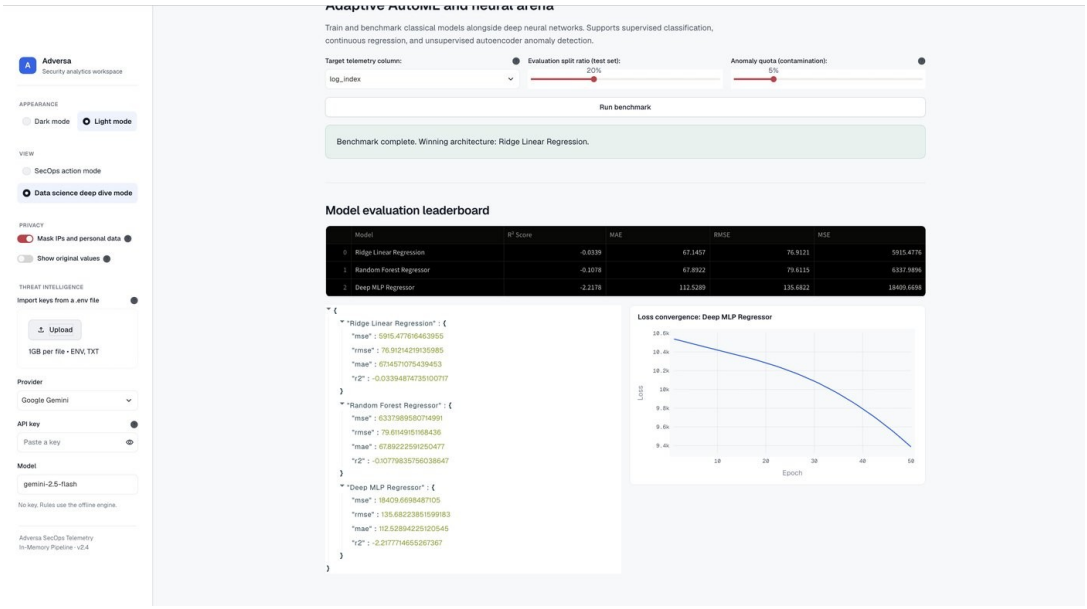


Figure 10. Regression mode on a meaningless target: all R-squared scores are at or below zero, as expected.

5.4 Adversarial robustness

The robustness tab attacks the winning model. It adds small, deliberate changes to the inputs and re-measures detection. The change grows from zero to a maximum size (epsilon, up to 0.3 on the chart). The evasion resistance score is the share of detection performance that survives. A flat line means the model is hard to fool. A steep fall means a small change is enough to evade it.

Run	Evasion resistance	Reading
Phishing, XGBoost	93.1%	Perturbations up to 20% noise cost only 6.9% of detection. Rated resilient.
Authentication log, autoencoder	68.7%	Noise reduced detection by 31.3%. The app rates this elevated risk and recommends adversarial retraining.

The second result is the more useful one to read. It shows the score is not always flattering, and it points to the kind of model that needs hardening. A small unsupervised model trained on 250 lines has little data to learn a firm boundary from.

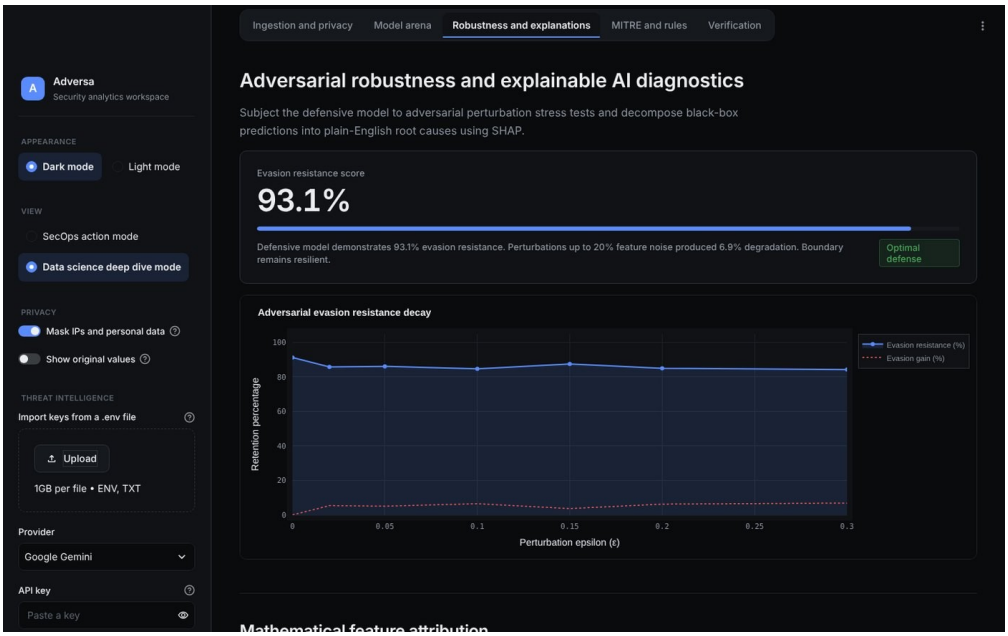


Figure 11. Phishing run: 93.1% evasion resistance, rated resilient.

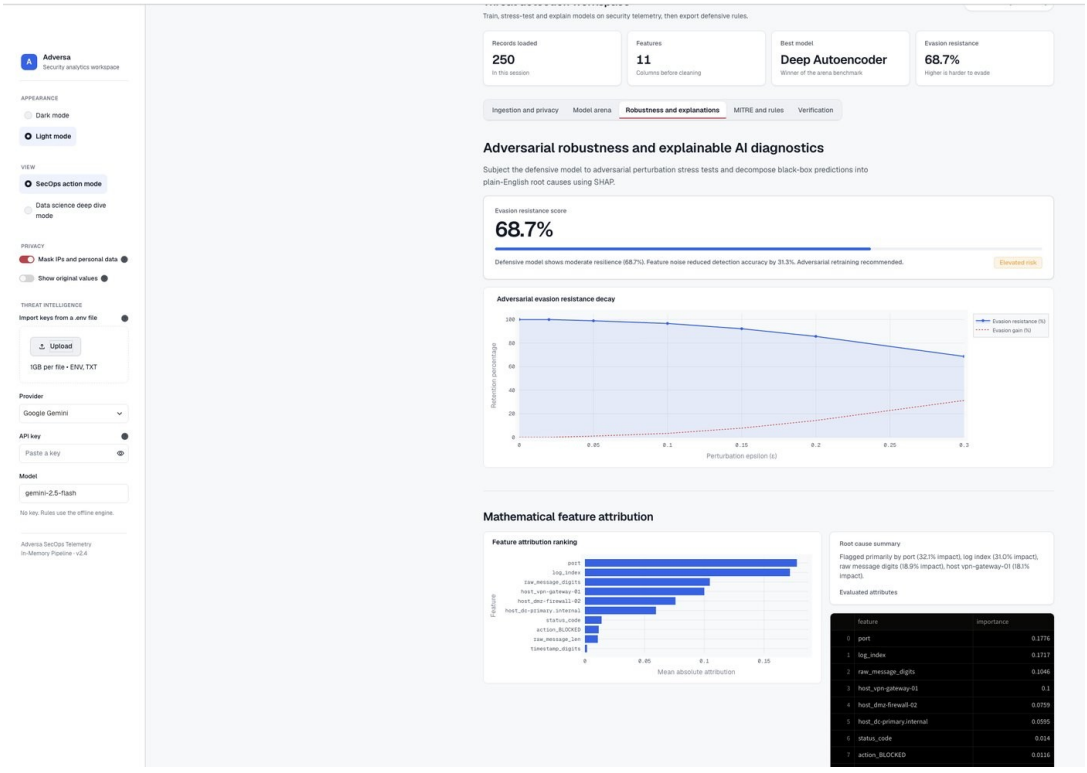


Figure 12. Authentication log run: 68.7% evasion resistance, rated elevated risk, with the decay curve and SHAP ranking below.

5.5 Explainability

SHAP assigns each feature a share of the credit for the model's decisions. The app shows the ranking as a bar chart and as a table, and writes a root-cause sentence. For the authentication log the top features were port (0.1776), log_index (0.1717), the number of digits in the raw message (0.1046), and the host being vpn-gateway-01 (0.1).

Two of these deserve care. The port is plausible, because failed attempts come from many different high source ports while accepted logins reuse port 52411. But log_index is only the row number. That it ranks second suggests the model has picked up an ordering artefact in this small

sample, not real behaviour. A user who reads the SHAP chart can spot this. A model that only reports accuracy would hide it.

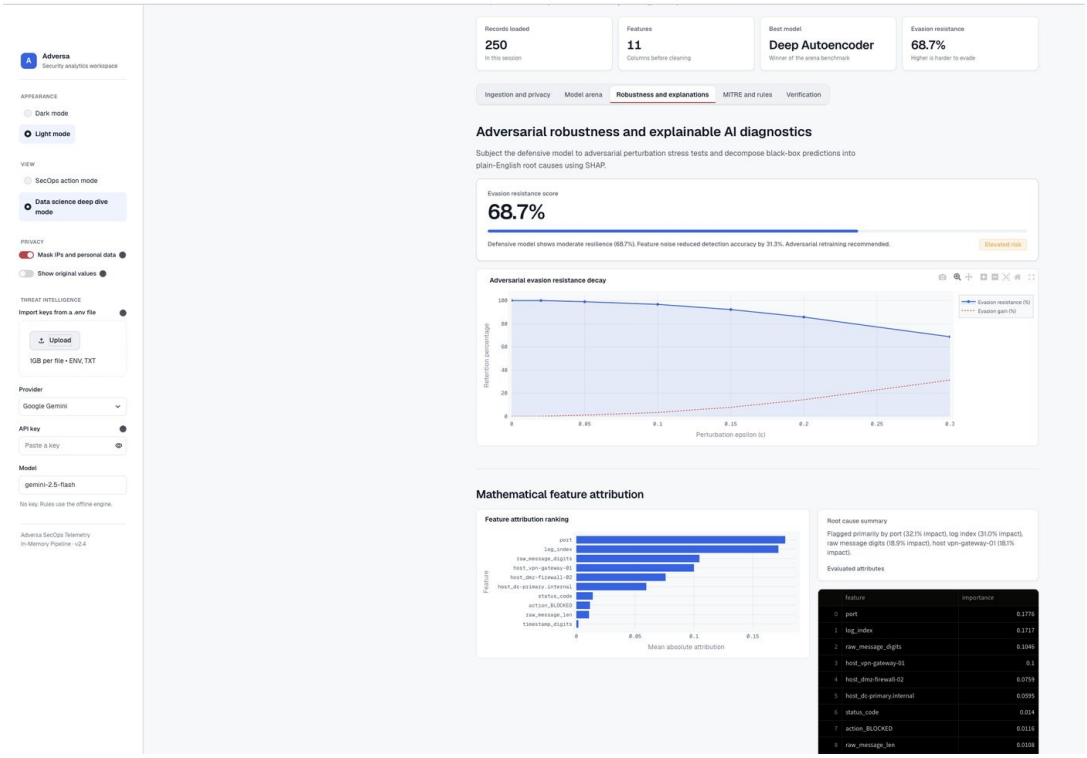


Figure 13. Data-science view of the same run: evasion decay curve and feature attribution ranking.

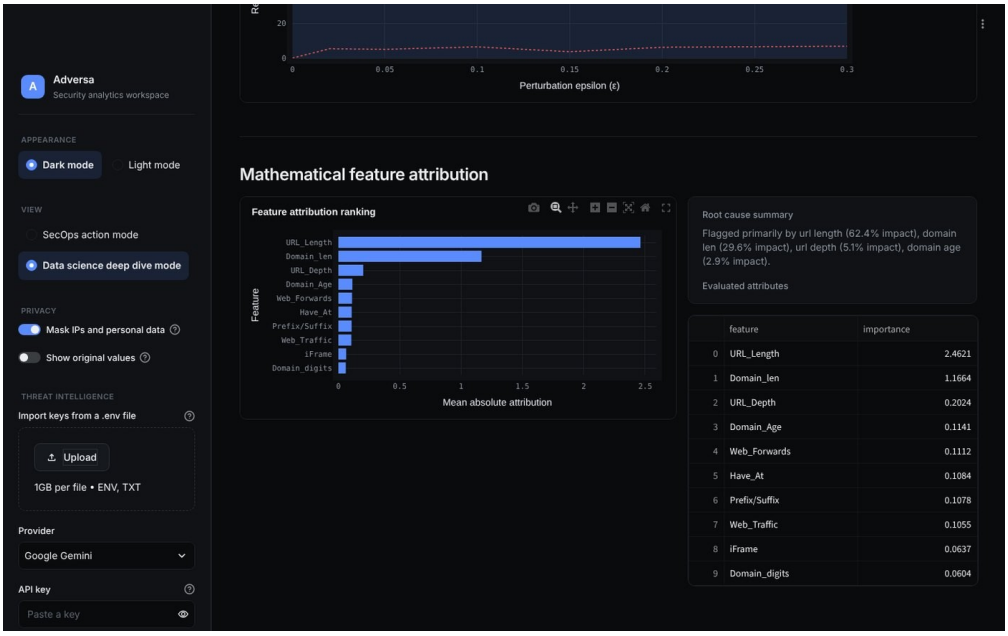


Figure 14. Phishing run: the top features driving the decisions are URL_Length and Domain_Len.

5.6 MITRE ATT&CK mapping and rule generation

The last analysis step turns findings into something a defender can use. For the log the offline generator mapped the behaviour to two techniques: T1046 Network Service Discovery (Discovery) for anomalous port activity, and T1110 Brute Force (Credential Access) for the repeated failed logins. That second mapping matches what the autoencoder flagged. It then wrote a YARA rule, a Snort signature, iptables commands and a remediation plan.

No API key was set, so the app used its built-in offline engine, and the page states this. With a key for Gemini, Claude, OpenAI or another provider, an LLM writes the rules instead. The offline rules are heuristic templates filled with the model's top features. They are a starting point for an analyst to review, not production signatures. For example, the YARA string "port" would match far too broadly if deployed as written.

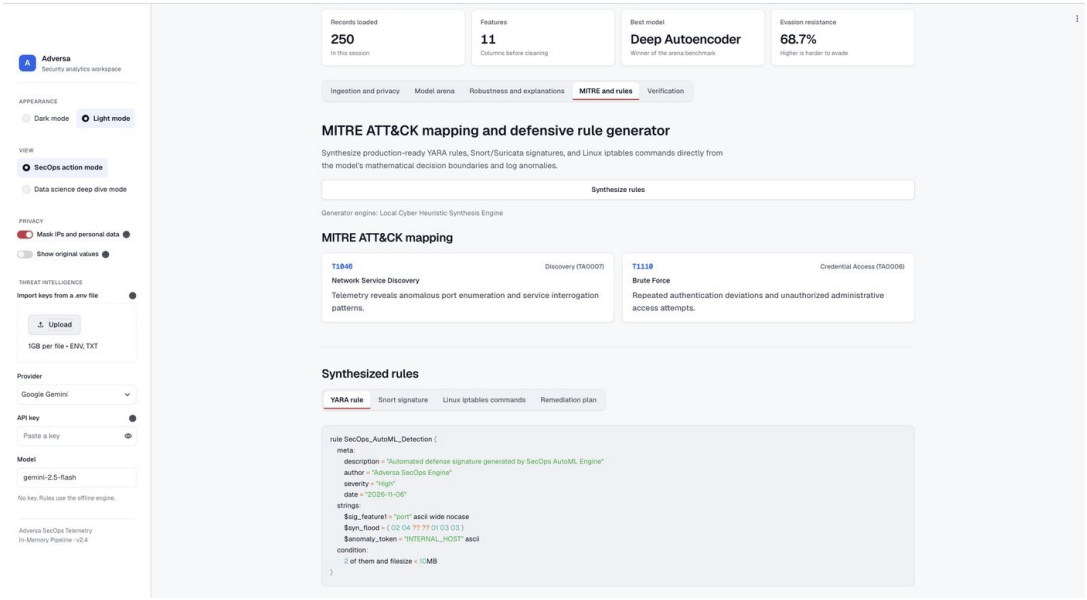


Figure 15. MITRE mapping (T1046, T1110) and the generated YARA rule.

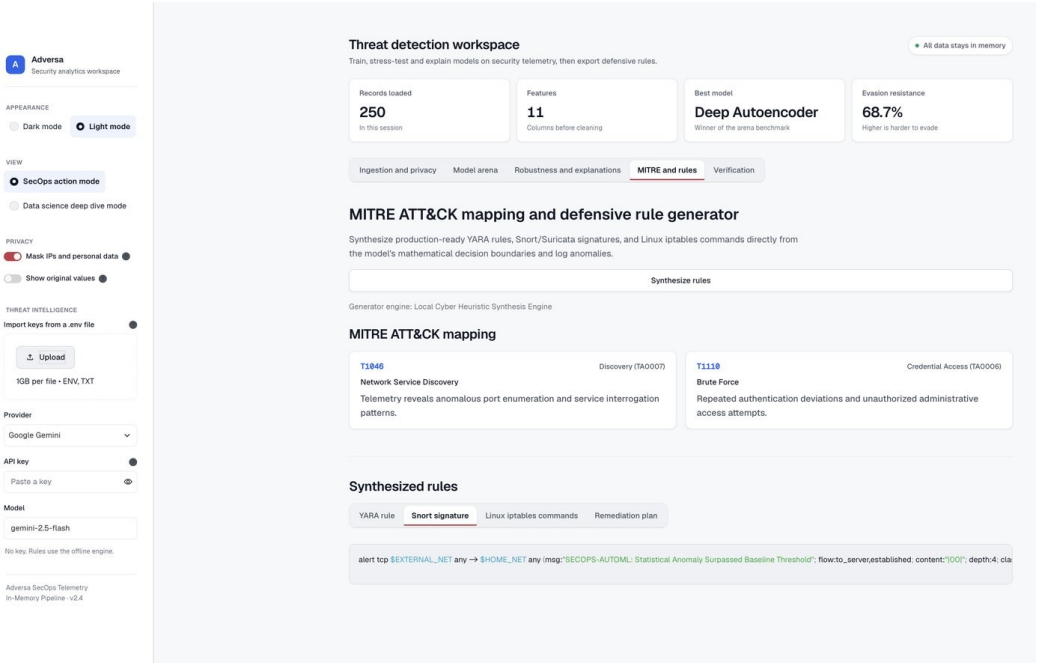


Figure 16. The generated Snort signature.

5.7 Verification

The Verification tab runs ten automated checks, one for each part of the system: the upload validator, privacy masking, multi-format ingestion, cleaning, the supervised arena, the autoencoder, the adversarial tester, SHAP, and two checks on the REST API. All ten passed in each run (10/10, 100% coverage, about 350 to 424 ms in total). The memory status reads 0 disk writes, confirming that nothing was saved to disk. The same checks run from the command line with pytest.

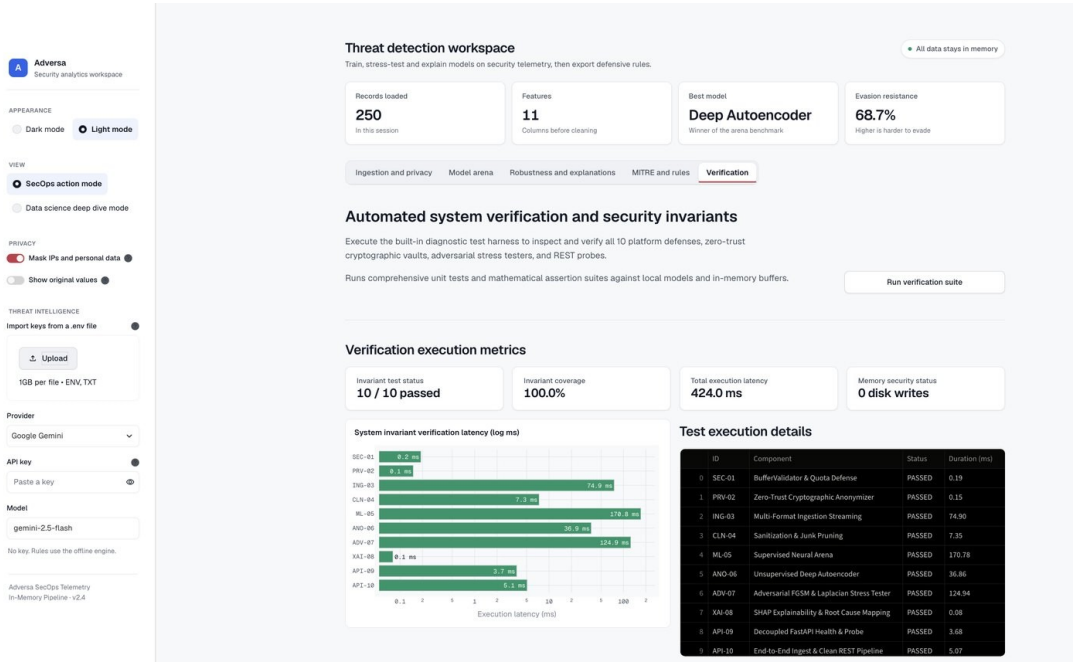


Figure 17. Verification suite: 10 of 10 invariants passed.

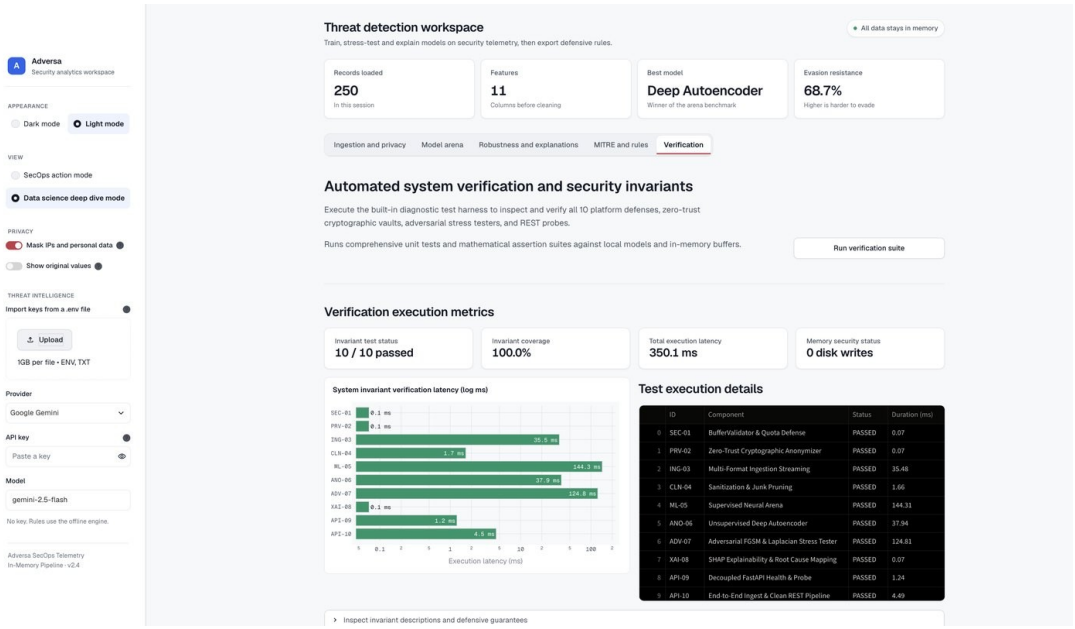


Figure 18. Verification in data-science view, with per-check latency.

6. Results summary

	Phishing URLs (main)	Authentication log (second)
Task	Supervised classification	Unsupervised anomaly detection
Records after cleaning	4,374 (from 10,000)	250
Winning model	XGBoost	Deep autoencoder
Detection result	93.5% catch rate, 6.6% false positives	13 of 250 entries flagged (5% quota)
Evasion resistance	93.1% (resilient)	68.7% (elevated risk)
Top explanation	URL length and domain length	Port, row index, message digits

	Phishing URLs (main)	Authentication log (second)
MITRE techniques	Phishing-related rules	T1046, T1110
Verification	10 of 10 passed	10 of 10 passed

7. Limitations and honest observations

- **Run-to-run variation.** Training is random, so the winner and the scores move a little between runs. The phishing evasion score was near 88% in one run and 93.1% in the recorded one.
- **The log sample is small and synthetic.** 250 lines are too few for strong conclusions. The 13 outliers are 5% of 250 because that was the quota I set, so it is not a measured detection rate.
- **Ordering artefact.** log_index ranking high in SHAP shows the model used the row number. In a real project I would drop identifier and index columns before training.
- **Unsupervised results have no ground truth.** Nothing in the log is labelled, so I checked the flagged lines by reading them. They are brute-force attempts, but this was a manual judgement.
- **Phishing features are pre-extracted.** The dataset gives 16 binary features, not the raw pages, so the model cannot see features that were not extracted.
- **Rules are heuristic.** Offline rules need analyst review before use. The adversarial test also uses simple perturbations, not every attack a real adversary could try.
- **Public duplicates.** The phishing file contained 56% duplicate rows. Without cleaning, its scores would be overstated.

8. What I learned

- **Prompt specificity decides the quality.** My first, short interface prompt produced a cluttered, hard-to-use page. A prompt with exact layout rules, a colour system, and a list of things to leave out produced a clean dashboard.
- **AI-written code has to be reviewed.** Reviewing the output found a slider that displayed 0%, results that disappeared when I switched view mode (the upload was being re-parsed on every refresh), a wrong run command in the README, and a light theme that left some widgets dark. Each was fixed and re-tested.
- **Accuracy alone is not enough.** The robustness score and the SHAP ranking gave information that the accuracy number did not, including a warning sign in the log run.
- **Clean the data first.** More than half of the public phishing rows were duplicates.
- **Tests give confidence.** With ten automated checks I could change the interface and the pipeline and see straight away that nothing broke.

9. Conclusion

Adversa meets the assignment brief. It is a working application, built with an AI IDE, that ingests a public cybersecurity dataset, cleans and masks it, compares models, attacks the best one, explains it, writes defensive rules and verifies itself. It also works on a different data type without

code changes, which was the goal of making it general. The main lessons were to be specific with the AI, to review what it writes, and to test the result.

10. Key prompts used

The full prompts are saved in the project (ANTIGRAVITY_PROMPT.md, ADVERSA_PROMPT.md, KEY_PROMPTS.md). The main ones:

#	Purpose	Summary
1	Build the app	Streamlit security-ML workspace: ingest CSV, JSON, PCAP, logs and Parquet; mask personal data in memory; compare several models; attack the best with adversarial inputs; explain with SHAP; generate MITRE-mapped rules.
2	Modern dashboard UI	Flat surfaces with one thin border, KPI cards across the top, left sidebar for settings, dark and light themes from one set of design tokens, no emoji, glow, glass or gradients.
3	Fix and refine	Make the interface look like a modern app with dashboards; remove AI-style decoration; add API-key import from many providers.
4	Rename	Rename the product from its earlier name to Adversa.
5	Dataset and demo	Use a public phishing dataset with its source cited; record a 2 to 5 minute demo covering the whole flow.

11. References

- Shreya Gopal. Phishing Website Detection by Machine Learning Techniques. github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques
- PhishTank. phishtank.com/developer_info.php
- Canadian Institute for Cybersecurity, University of New Brunswick. URL 2016 dataset. unb.ca/cic/datasets/url-2016.html
- UCI Machine Learning Repository. Phishing Websites. archive.ics.uci.edu/ml/datasets/Phishing+Websites
- MITRE ATT&CK. attack.mitre.org (T1046 Network Service Discovery, T1110 Brute Force)
- Lundberg and Lee. A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS 2017
- Goodfellow, Shlens and Szegedy. Explaining and Harnessing Adversarial Examples (FGSM). ICLR 2015